Spring JPA Exercise 1: Basic JPA Entity and Repository

Objectives

Explain the need and benefit of ORM

ORM (Object-Relational Mapping), makes it easier t- develop code that interacts with database, abstracts the database system, transactionality

ORM Pros and Cons - https://blog.bitsrc.io/what-is-an-orm-and-why-you-should-use-it-b2b6f75f5e2a

What is ORM? - https://en.wikipedia.org/wiki/Object-relational_mapping

Demonstrate the need and benefit of Spring Data JPA

Evolution of ORM solutions, Hibernate XML Configuration, Hibernate Annotation Configuration, Spring Data JPA, Hibernate benefits, open source, light weight, database independent query

With H2 in memory database - https://www.mkyong.com/spring-boot/spring-boot-spring-data-jpa/

With MySQL - https://www.mkyong.com/spring-boot/spring-boot-spring-data-jpa-mysql-example/

XML Configuration Example -https://www.tutorialspoint.com/hibernate/hibernate_examples.htm

Hibernate Configuration Example -https://www.tutorialspoint.com/hibernate/hibernate_annotations.htm

Explain about core objects of hibernate framework

Session Factory, Session, Transaction Factory, Transaction, Connection Provider

Hibernate Architecture Reference - https://www.tutorialspoint.com/hibernate/hibernate_architecture.htm

Explain ORM implementation with Hibernate XML Configuration and Annotation Configuration

XML Configuration - persistence class, mapping xml, configuration xml, loading hibernate configuration xml file; Annotation Configuration - persistence class, @Entity, @Table, @Id, @Column, hibernate configuration xml file Loading hibernate configuration and interacting with database get the session factory, open session, begin transaction, commit transaction, close session

XML Configuration Example - https://www.tutorialspoint.com/hibernate/hibernate_examples.htm

Hibernate Configuration Example - https://www.tutorialspoint.com/hibernate/hibernate_annotations.htm

Explain the difference between Java Persistence API, Hibernate and Spring Data JPA

JPA (Java Persistence API), JPA is a specification (JSR 338), JPA does not have implementation, Hibernate is one of the implementation for JPA, Hibernate is a ORM tool, Spring Data JPA is an abstraction above Hibernate t- remove boiler plate code when persisting data using Hibernate.

Difference between Spring Data JPA and Hibernate - https://dzone.com/articles/what-is-the-difference-between-hibernate-and-sprin-1

Intr- t- JPA - https://www.javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html

Demonstrate implementation of DML using Spring Data JPA on a single database table

Hibernate log configuration and ddl-aut- configuration, JpaRepsitory.findById(), defining Query Methods, JpaRespository.save(), JpaRepository.deleteById()

Spring Data JPA Ref Repository methods - https://docs.spring.io/spring-data/jpa/docs/2.2.0.RELEASE/reference/html/#repositories.core-concepts

Spring JPA Exercise 1: Basic JPA Entity and Repository

Query methods - https://docs.spring.io/spring-data/jpa/docs/2.2.0.RELEASE/reference/html/#repositories.query-methods

Hands on 1

Spring Data JPA - Quick Example

Software Pre-requisites

MySQL Server 8.0

MySQL Workbench 8

Eclipse IDE for Enterprise Java Developers 2019-03 R

Maven 3.6.2

Create a Eclipse Project using Spring Initializr

G- to https://start.spring.io/

Change Group as "com.cognizant"

Change Artifact Id as "orm-learn"

In Options > Description enter "Dem- project for Spring Data JPA and Hibernate"

Click on menu and select "Spring Boot DevTools", "Spring Data JPA" and "MySQL Driver"

Click Generate and download the project as zip

Extract the zip in root folder t- Eclipse Workspace

Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"

Create a new schema "ormlearn" in MySQL database. Execute the following commands t- open MySQL client and create schema.

> mysql -u root -p

mysql> create schema ormlearn;

In orm-learn Eclipse project, open src/main/resources/application.properties and include the below database and log configuration.

```
# Spring Framework and application log
logging.level.org.springframework=info
logging.level.com.cognizant=debug
# Hibernate logs for displaying executed SQL, input and output
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace
# Log pattern
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-25.25logger{25} %25M %4L %m%n
# Database configuration
```

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn

spring.datasource.username=root

spring.datasource.password=root

Hibernate configuration

spring.jpa.hibernate.ddl-auto=validate

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect

Build the project using ?mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456' command in command line

Include logs for verifying if main() method is called.

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class);

public static void main(String[] args) {

SpringApplication.run(OrmLearnApplication.class, args);

LOGGER.info("Inside main");

}

Execute the OrmLearnApplication and check in log if main method is called.

SME t- walk through the following aspects related t- the project created:

src/main/java - Folder with application code

src/main/resources - Folder for application configuration

src/test/java - Folder with code for testing the application

OrmLearnApplication.java - Walkthrough the main() method.

Purpose of @SpringBootApplication annotation

pom.xml

Walkthrough all the configuration defined in XML file

Open 'Dependency Hierarchy' and show the dependency tree.

Country table creation

Create a new table country with columns for code and name. For sample, let us insert one country with values 'IN' and 'India' in this table.

create table country(co_code varchar(2) primary key, co_name varchar(50));

Insert couple of records int- the table

insert int- country values ('IN', 'India');

insert int- country values ('US', 'United States of America');

Persistence Class - com.cognizant.orm-learn.model.Country

Spring JPA Exercise 1: Basic JPA Entity and Repository

Open Eclipse with orm-learn project

Create new package com.cognizant.orm-learn.model

Create Country.java, then generate getters, setters and toString() methods.

Include @Entity and @Table at class level

Include @Column annotations in each getter method specifying the column name.

```
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="country")
public class Country {
@Id
@Column(name="code")
private String code;
@Column(name="name")
private String name;
// getters and setters
// toString()
}
```

Notes:

@Entity is an indicator t- Spring Data JPA that it is an entity class for the application

@Table helps in defining the mapping database table

@Id helps is defining the primary key

@Column helps in defining the mapping table column

Repository Class - com.cognizant.orm-learn.CountryRepository

Create new package com.cognizant.orm-learn.repository

Create new interface named CountryRepository that extends JpaRepository<Country, String>

Define @Repository annotation at class level

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}
```

Service Class - com.cognizant.orm-learn.service.CountryService

Spring JPA Exercise 1: Basic JPA Entity and Repository

Create new package com.cognizant.orm-learn.service

Create new class CountryService

Include @Service annotation at class level

Autowire CountryRepository in CountryService

Include new method getAllCountries() method that returns a list of countries.

Include @Transactional annotation for this method

In getAllCountries() method invoke countryRepository.findAll() method and return the result

Testing in OrmLearnApplication.java

Include a static reference t- CountryService in OrmLearnApplication class

private static CountryService countryService;

Define a test method t- get all countries from service.

private static void testGetAllCountries() {

LOGGER.info("Start");

List<Country> countries = countryService.getAllCountries();

LOGGER.debug("countries={}", countries);

LOGGER.info("End");

}

Modify SpringApplication.run() invocation t- set the application context and the CountryService reference from the application context.

ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);

countryService = context.getBean(CountryService.class);

testGetAllCountries();

Execute main method t- check if data from ormlearn database is retrieved.

Hands on 2

Hibernate XML Config implementation walk through

SME t- provide explanation on the sample Hibernate implementation available in the link below:

https://www.tutorialspoint.com/hibernate/hibernate_examples.htm

Explanation Topics

Explain how object t- relational database mapping done in hibernate xml configuration file

Explain about following aspects of implementing the end t- end operations in Hibernate:

SessionFactory

Session

Transaction

beginTransaction()

commit()

rollback()

session.save()

session.createQuery().list()

session.get()

session.delete()

Hands on 3

Hibernate Annotation Config implementation walk through

SME t- provide explanation on the sample Hibernate implementation available in the link below:

https://www.tutorialspoint.com/hibernate/hibernate_annotations.htm

Explanation Topics

Explain how object t- relational database mapping done in persistence class file Employee

Explain about following aspects of implementing the end t- end operations in Hibernate:

@Entity

@Table

@Id

@GeneratedValue

@Column

Hibernate Configuration (hibernate.cfg.xml)

Dialect

Driver

Connection URL

Username

Password

Hands on 4

Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

JSR 338 Specification for persisting, reading and managing data from Java objects

Does not contain concrete implementation of the specification

Hibernate is one of the implementation of JPA

Hibernate

ORM Tool that implements JPA

Spring Data JPA

Spring JPA Exercise 1: Basic JPA Entity and Repository

Does not have JPA implementation, but reduces boiler plate code

This is another level of abstraction over JPA implementation provider like Hibernate

Manages transactions

Refer code snippets below on how the code compares between Hibernate and Spring Data JPA

Hibernate

```
/* Method t- CREATE an employee in the database */
public Integer addEmployee(Employee employee){
Session session = factory.openSession();
Transaction tx = null;
Integer employeeID = null;
try {
tx = session.beginTransaction();
employeeID = (Integer) session.save(employee);
tx.commit();
} catch (HibernateException e) {
if (tx != null) tx.rollback();
e.printStackTrace();
} finally {
session.close();
}
return employeeID;
}
```

Spring Data JPA

EmployeeRespository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

EmployeeService.java

```
@Autowire
private EmployeeRepository employeeRepository;
@Transactional
public void addEmployee(Employee employee) {
employeeRepository.save(employee);
}
```

???????

Reference Links:

Spring JPA Exercise 1: Basic JPA Entity and Repository

https://dzone.com/articles/what-is-the-difference-between-hibernate-and-sprin-1

https://www.javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html

Hands on 5

Implement services for managing Country

An application requires for features t- be implemented with regards t- country. These features needs t- be supported by implementing them as service using Spring Data JPA.

Find a country based on country code

Add new country

Update country

Delete country

Find list of countries matching a partial country name

Before starting the implementation of the above features, there are few configuration and data population that needs t- be incorporated. Please refer each topic below and implement the same.

Explanation for Hibernate table creation configuration

Moreover the ddl-aut- defines how hibernate behaves if a specific table or column is not present in the database.

create - drops existing tables data and structure, then creates new tables

validate - check if the table and columns exist or not, throws an exception if a matching table or column is not found

update - if a table does not exists, it creates a new table; if a column does not exists, it creates a new column

create-drop - creates the table, once all operations are completed, the table is dropped

Hibernate ddl aut- (create, create-drop, update, validate)

spring.jpa.hibernate.ddl-auto=validate

Populate country table

Delete all the records in Country table and then use the below script t- create the actual list of all countries in our world.

insert int- country (co_code, co_name) values ("AF", "Afghanistan");

insert int- country (co_code, co_name) values ("AL", "Albania");

insert int- country (co_code, co_name) values ("DZ", "Algeria");

insert int- country (co_code, co_name) values ("AS", "American Samoa");

insert int- country (co_code, co_name) values ("AD", "Andorra");

insert int- country (co_code, co_name) values ("AO", "Angola");

insert int- country (co_code, co_name) values ("AI", "Anguilla");

insert int- country (co_code, co_name) values ("AQ", "Antarctica");

insert int- country (co_code, co_name) values ("AG", "Antigua and Barbuda");
insert int- country (co_code, co_name) values ("AR", "Argentina");
insert int- country (co_code, co_name) values ("AM", "Armenia");
insert int- country (co_code, co_name) values ("AW", "Aruba");
insert int- country (co_code, co_name) values ("AU", "Australia");
insert int- country (co_code, co_name) values ("AT", "Austria");
insert int- country (co_code, co_name) values ("AZ", "Azerbaijan");
insert int- country (co_code, co_name) values ("BS", "Bahamas");
insert int- country (co_code, co_name) values ("BH", "Bahrain");
insert int- country (co_code, co_name) values ("BD", "Bangladesh");
insert int- country (co_code, co_name) values ("BB", "Barbados");
insert int- country (co_code, co_name) values ("BY", "Belarus");
insert int- country (co_code, co_name) values ("BE", "Belgium");
insert int- country (co_code, co_name) values ("BZ", "Belize");
insert int- country (co_code, co_name) values ("BJ", "Benin");
insert int- country (co_code, co_name) values ("BM", "Bermuda");
insert int- country (co_code, co_name) values ("BT", "Bhutan");
insert int- country (co_code, co_name) values ("BO", "Bolivia, Plurinational State of");
insert int- country (co_code, co_name) values ("BQ", "Bonaire, Sint Eustatius and Saba");
insert int- country (co_code, co_name) values ("BA", "Bosnia and Herzegovina");
insert int- country (co_code, co_name) values ("BW", "Botswana");
insert int- country (co_code, co_name) values ("BV", "Bouvet Island");
insert int- country (co_code, co_name) values ("BR", "Brazil");
insert int- country (co_code, co_name) values ("IO", "British Indian Ocean Territory");
insert int- country (co_code, co_name) values ("BN", "Brunei Darussalam");
insert int- country (co_code, co_name) values ("BG", "Bulgaria");
insert int- country (co_code, co_name) values ("BF", "Burkina Faso");
insert int- country (co_code, co_name) values ("BI", "Burundi");
insert int- country (co_code, co_name) values ("KH", "Cambodia");
insert int- country (co_code, co_name) values ("CM", "Cameroon");
insert int- country (co_code, co_name) values ("CA", "Canada");
insert int- country (co_code, co_name) values ("CV", "Cape Verde");
insert int- country (co_code, co_name) values ("KY", "Cayman Islands");
insert int- country (co_code, co_name) values ("CF", "Central African Republic");
insert int- country (co_code, co_name) values ("TD", "Chad");
insert int- country (co_code, co_name) values ("CL", "Chile");

insert int- country (co_code, co_name) values ("CN", "China");
insert int- country (co_code, co_name) values ("CX", "Christmas Island");
insert int- country (co_code, co_name) values ("CC", "Cocos (Keeling) Islands");
insert int- country (co_code, co_name) values ("CO", "Colombia");
insert int- country (co_code, co_name) values ("KM", "Comoros");
insert int- country (co_code, co_name) values ("CG", "Congo");
insert int- country (co_code, co_name) values ("CD", "Congo, the Democratic Republic of the");
insert int- country (co_code, co_name) values ("CK", "Cook Islands");
insert int- country (co_code, co_name) values ("CR", "Costa Rica");
insert int- country (co_code, co_name) values ("HR", "Croatia");
insert int- country (co_code, co_name) values ("CU", "Cuba");
insert int- country (co_code, co_name) values ("CW", "Curaçao");
insert int- country (co_code, co_name) values ("CY", "Cyprus");
insert int- country (co_code, co_name) values ("CZ", "Czech Republic");
insert int- country (co_code, co_name) values ("CI", "Côte d'Ivoire");
insert int- country (co_code, co_name) values ("DK", "Denmark");
insert int- country (co_code, co_name) values ("DJ", "Djibouti");
insert int- country (co_code, co_name) values ("DM", "Dominica");
insert int- country (co_code, co_name) values ("DO", "Dominican Republic");
insert int- country (co_code, co_name) values ("EC", "Ecuador");
insert int- country (co_code, co_name) values ("EG", "Egypt");
insert int- country (co_code, co_name) values ("SV", "El Salvador");
insert int- country (co_code, co_name) values ("GQ", "Equatorial Guinea");
insert int- country (co_code, co_name) values ("ER", "Eritrea");
insert int- country (co_code, co_name) values ("EE", "Estonia");
insert int- country (co_code, co_name) values ("ET", "Ethiopia");
insert int- country (co_code, co_name) values ("FK", "Falkland Islands (Malvinas)");
insert int- country (co_code, co_name) values ("FO", "Faroe Islands");
insert int- country (co_code, co_name) values ("FJ", "Fiji");
insert int- country (co_code, co_name) values ("FI", "Finland");
insert int- country (co_code, co_name) values ("FR", "France");
insert int- country (co_code, co_name) values ("GF", "French Guiana");
insert int- country (co_code, co_name) values ("PF", "French Polynesia");
insert int- country (co_code, co_name) values ("TF", "French Southern Territories");
insert int- country (co_code, co_name) values ("GA", "Gabon");
insert int- country (co_code, co_name) values ("GM", "Gambia");

insert int- country (co_code, co_name) values ("GE", "Georgia");
insert int- country (co_code, co_name) values ("DE", "Germany");
insert int- country (co_code, co_name) values ("GH", "Ghana");
insert int- country (co_code, co_name) values ("GI", "Gibraltar");
insert int- country (co_code, co_name) values ("GR", "Greece");
insert int- country (co_code, co_name) values ("GL", "Greenland");
insert int- country (co_code, co_name) values ("GD", "Grenada");
insert int- country (co_code, co_name) values ("GP", "Guadeloupe");
insert int- country (co_code, co_name) values ("GU", "Guam");
insert int- country (co_code, co_name) values ("GT", "Guatemala");
insert int- country (co_code, co_name) values ("GG", "Guernsey");
insert int- country (co_code, co_name) values ("GN", "Guinea");
insert int- country (co_code, co_name) values ("GW", "Guinea-Bissau");
insert int- country (co_code, co_name) values ("GY", "Guyana");
insert int- country (co_code, co_name) values ("HT", "Haiti");
insert int- country (co_code, co_name) values ("HM", "Heard Island and McDonald Islands");
insert int- country (co_code, co_name) values ("VA", "Holy See (Vatican City State)");
insert int- country (co_code, co_name) values ("HN", "Honduras");
insert int- country (co_code, co_name) values ("HK", "Hong Kong");
insert int- country (co_code, co_name) values ("HU", "Hungary");
insert int- country (co_code, co_name) values ("IS", "Iceland");
insert int- country (co_code, co_name) values ("IN", "India");
insert int- country (co_code, co_name) values ("ID", "Indonesia");
insert int- country (co_code, co_name) values ("IR", "Iran, Islamic Republic of");
insert int- country (co_code, co_name) values ("IQ", "Iraq");
insert int- country (co_code, co_name) values ("IE", "Ireland");
insert int- country (co_code, co_name) values ("IM", "Isle of Man");
insert int- country (co_code, co_name) values ("IL", "Israel");
insert int- country (co_code, co_name) values ("IT", "Italy");
insert int- country (co_code, co_name) values ("JM", "Jamaica");
insert int- country (co_code, co_name) values ("JP", "Japan");
insert int- country (co_code, co_name) values ("JE", "Jersey");
insert int- country (co_code, co_name) values ("JO", "Jordan");
insert int- country (co_code, co_name) values ("KZ", "Kazakhstan");
insert int- country (co_code, co_name) values ("KE", "Kenya");
insert int- country (co_code, co_name) values ("KI", "Kiribati");

insert int- country (co_code, co_name) values ("KP", "Democratic People's Republic of Korea");
insert int- country (co_code, co_name) values ("KR", "Republic of Korea");
insert int- country (co_code, co_name) values ("KW", "Kuwait");
insert int- country (co_code, co_name) values ("KG", "Kyrgyzstan");
insert int- country (co_code, co_name) values ("LA", "La- People's Democratic Republic");
insert int- country (co_code, co_name) values ("LV", "Latvia");
insert int- country (co_code, co_name) values ("LB", "Lebanon");
insert int- country (co_code, co_name) values ("LS", "Lesotho");
insert int- country (co_code, co_name) values ("LR", "Liberia");
insert int- country (co_code, co_name) values ("LY", "Libya");
insert int- country (co_code, co_name) values ("LI", "Liechtenstein");
insert int- country (co_code, co_name) values ("LT", "Lithuania");
insert int- country (co_code, co_name) values ("LU", "Luxembourg");
insert int- country (co_code, co_name) values ("MO", "Macao");
insert int- country (co_code, co_name) values ("MK", "Macedonia, the Former Yugoslav Republic of");
insert int- country (co_code, co_name) values ("MG", "Madagascar");
insert int- country (co_code, co_name) values ("MW", "Malawi");
insert int- country (co_code, co_name) values ("MY", "Malaysia");
insert int- country (co_code, co_name) values ("MV", "Maldives");
insert int- country (co_code, co_name) values ("ML", "Mali");
insert int- country (co_code, co_name) values ("MT", "Malta");
insert int- country (co_code, co_name) values ("MH", "Marshall Islands");
insert int- country (co_code, co_name) values ("MQ", "Martinique");
insert int- country (co_code, co_name) values ("MR", "Mauritania");
insert int- country (co_code, co_name) values ("MU", "Mauritius");
insert int- country (co_code, co_name) values ("YT", "Mayotte");
insert int- country (co_code, co_name) values ("MX", "Mexico");
insert int- country (co_code, co_name) values ("FM", "Micronesia, Federated States of");
insert int- country (co_code, co_name) values ("MD", "Moldova, Republic of");
insert int- country (co_code, co_name) values ("MC", "Monaco");
insert int- country (co_code, co_name) values ("MN", "Mongolia");
insert int- country (co_code, co_name) values ("ME", "Montenegro");
insert int- country (co_code, co_name) values ("MS", "Montserrat");
insert int- country (co_code, co_name) values ("MA", "Morocco");
insert int- country (co_code, co_name) values ("MZ", "Mozambique");
insert int- country (co_code, co_name) values ("MM", "Myanmar");

insert int- country (co_code, co_name) values ("NA", "Namibia");
insert int- country (co_code, co_name) values ("NR", "Nauru");
insert int- country (co_code, co_name) values ("NP", "Nepal");
insert int- country (co_code, co_name) values ("NL", "Netherlands");
insert int- country (co_code, co_name) values ("NC", "New Caledonia");
insert int- country (co_code, co_name) values ("NZ", "New Zealand");
insert int- country (co_code, co_name) values ("NI", "Nicaragua");
insert int- country (co_code, co_name) values ("NE", "Niger");
insert int- country (co_code, co_name) values ("NG", "Nigeria");
insert int- country (co_code, co_name) values ("NU", "Niue");
insert int- country (co_code, co_name) values ("NF", "Norfolk Island");
insert int- country (co_code, co_name) values ("MP", "Northern Mariana Islands");
insert int- country (co_code, co_name) values ("NO", "Norway");
insert int- country (co_code, co_name) values ("OM", "Oman");
insert int- country (co_code, co_name) values ("PK", "Pakistan");
insert int- country (co_code, co_name) values ("PW", "Palau");
insert int- country (co_code, co_name) values ("PS", "Palestine, State of");
insert int- country (co_code, co_name) values ("PA", "Panama");
insert int- country (co_code, co_name) values ("PG", "Papua New Guinea");
insert int- country (co_code, co_name) values ("PY", "Paraguay");
insert int- country (co_code, co_name) values ("PE", "Peru");
insert int- country (co_code, co_name) values ("PH", "Philippines");
insert int- country (co_code, co_name) values ("PN", "Pitcairn");
insert int- country (co_code, co_name) values ("PL", "Poland");
insert int- country (co_code, co_name) values ("PT", "Portugal");
insert int- country (co_code, co_name) values ("PR", "Puert- Rico");
insert int- country (co_code, co_name) values ("QA", "Qatar");
insert int- country (co_code, co_name) values ("RO", "Romania");
insert int- country (co_code, co_name) values ("RU", "Russian Federation");
insert int- country (co_code, co_name) values ("RW", "Rwanda");
insert int- country (co_code, co_name) values ("RE", "Réunion");
insert int- country (co_code, co_name) values ("BL", "Saint Barthélemy");
insert int- country (co_code, co_name) values ("SH", "Saint Helena, Ascension and Tristan da Cunha");
insert int- country (co_code, co_name) values ("KN", "Saint Kitts and Nevis");
insert int- country (co_code, co_name) values ("LC", "Saint Lucia");
insert int- country (co_code, co_name) values ("MF", "Saint Martin (French part)");

insert int- country (co_code, co_name) values ("PM", "Saint Pierre and Miquelon");
insert int- country (co_code, co_name) values ("VC", "Saint Vincent and the Grenadines");
insert int- country (co_code, co_name) values ("WS", "Samoa");
insert int- country (co_code, co_name) values ("SM", "San Marino");
insert int- country (co_code, co_name) values ("ST", "Sa- Tome and Principe");
insert int- country (co_code, co_name) values ("SA", "Saudi Arabia");
insert int- country (co_code, co_name) values ("SN", "Senegal");
insert int- country (co_code, co_name) values ("RS", "Serbia");
insert int- country (co_code, co_name) values ("SC", "Seychelles");
insert int- country (co_code, co_name) values ("SL", "Sierra Leone");
insert int- country (co_code, co_name) values ("SG", "Singapore");
insert int- country (co_code, co_name) values ("SX", "Sint Maarten (Dutch part)");
insert int- country (co_code, co_name) values ("SK", "Slovakia");
insert int- country (co_code, co_name) values ("SI", "Slovenia");
insert int- country (co_code, co_name) values ("SB", "Solomon Islands");
insert int- country (co_code, co_name) values ("SO", "Somalia");
insert int- country (co_code, co_name) values ("ZA", "South Africa");
insert int- country (co_code, co_name) values ("GS", "South Georgia and the South Sandwich Islands");
insert int- country (co_code, co_name) values ("SS", "South Sudan");
insert int- country (co_code, co_name) values ("ES", "Spain");
insert int- country (co_code, co_name) values ("LK", "Sri Lanka");
insert int- country (co_code, co_name) values ("SD", "Sudan");
insert int- country (co_code, co_name) values ("SR", "Suriname");
insert int- country (co_code, co_name) values ("SJ", "Svalbard and Jan Mayen");
insert int- country (co_code, co_name) values ("SZ", "Swaziland");
insert int- country (co_code, co_name) values ("SE", "Sweden");
insert int- country (co_code, co_name) values ("CH", "Switzerland");
insert int- country (co_code, co_name) values ("SY", "Syrian Arab Republic");
insert int- country (co_code, co_name) values ("TW", "Taiwan, Province of China");
insert int- country (co_code, co_name) values ("TJ", "Tajikistan");
insert int- country (co_code, co_name) values ("TZ", "Tanzania, United Republic of");
insert int- country (co_code, co_name) values ("TH", "Thailand");
insert int- country (co_code, co_name) values ("TL", "Timor-Leste");
insert int- country (co_code, co_name) values ("TG", "Togo");
insert int- country (co_code, co_name) values ("TK", "Tokelau");
insert int- country (co_code, co_name) values ("TO", "Tonga");

insert int- country (co_code, co_name) values ("TT", "Trinidad and Tobago");

insert int- country (co_code, co_name) values ("TN", "Tunisia");

insert int- country (co_code, co_name) values ("TR", "Turkey");

insert int- country (co_code, co_name) values ("TM", "Turkmenistan");

insert int- country (co_code, co_name) values ("TC", "Turks and Caicos Islands");

insert int- country (co_code, co_name) values ("TV", "Tuvalu");

insert int- country (co_code, co_name) values ("UG", "Uganda");

insert int- country (co_code, co_name) values ("UA", "Ukraine");

insert int- country (co_code, co_name) values ("AE", "United Arab Emirates");

insert int- country (co_code, co_name) values ("GB", "United Kingdom");

insert int- country (co_code, co_name) values ("US", "United States");

insert int- country (co_code, co_name) values ("UM", "United States Minor Outlying Islands");

insert int- country (co_code, co_name) values ("UY", "Uruguay");

insert int- country (co_code, co_name) values ("UZ", "Uzbekistan");

insert int- country (co_code, co_name) values ("VU", "Vanuatu");

insert int- country (co_code, co_name) values ("VE", "Venezuela, Bolivarian Republic of");

insert int- country (co_code, co_name) values ("VN", "Viet Nam");

insert int- country (co_code, co_name) values ("VG", "Virgin Islands, British");

insert int- country (co_code, co_name) values ("VI", "Virgin Islands, U.S.");

insert int- country (co_code, co_name) values ("WF", "Wallis and Futuna");

insert int- country (co_code, co_name) values ("EH", "Western Sahara");

insert int- country (co_code, co_name) values ("YE", "Yemen");

insert int- country (co_code, co_name) values ("ZM", "Zambia");

insert int- country (co_code, co_name) values ("ZW", "Zimbabwe");

insert int- country (co_code, co_name) values ("AX", "Åland Islands");

Refer subsequent hands on exercises t- implement the features related t- country.

Hands on 6

Find a country based on country code

Create new exception class CountryNotFoundException in com.cognizant.spring-learn.service.exception

Create new method findCountryByCode() in CountryService with @Transactional annotation

In findCountryByCode() method, perform the following steps:

Method signature

@Transactional

public Country findCountryByCode(String countryCode) throws CountryNotFoundException

Get the country based on findById() built in method

Optional<Country> result = countryRepository.findById(countryCode);

From the result, check if a country is found. If not found, throw CountryNotFoundException

if (!result.isPresent())

Use get() method t- return the country fetched.

Country country = result.get();

Include new test method in OrmLearnApplication t- find a country based on country code and compare the country name t- check if it is valid.

private static void getAllCountriesTest() {

LOGGER.info("Start");

Country country = countryService.findCountryByCode("IN");

LOGGER.debug("Country:{}", country);

LOGGER.info("End");

}

Invoke the above method in main() method and test it.

NOTE: SME t- explain the importance of @Transactional annotation. Spring takes care of creating the Hibernate session and manages the transactionality when executing the service method.

Hands on 7

Add a new country

Create new method in CountryService.

@Transactional

public void addCountry(Country country)

Invoke save() method of repository t- get the country added.

countryRepository.save(country)

Include new testAddCountry() method in OrmLearnApplication. Perform steps below:

Create new instance of country with a new code and name

Call countryService.addCountry() passing the country created in the previous step.

Invoke countryService.findCountryByCode() passing the same code used when adding a new country

Check in the database if the country is added

Hands on 8

Update a country based on code

Create a new method updateCountry() in CountryService with parameters code and name. Annotate this method with @Transactional. Implement following steps in this method.

Get the reference of the country using findById() method in repository

In the country reference obtained, update the name of country using setter method

Call countryRepository.save() method t- update the name

Include new test method in OrmLearnApplication, which invokes updateCountry() method in CountryService passing a country's code and different name for the country.

Spring JPA Exercise 1: Basic JPA Entity and Repository

Check in database table if name is modified.

Hands on 9

Delete a country based on code

Create new method deleteCountry() in CountryService. Annotate this method with @Transactional.

In deleteCountry() method call deleteById() method of repository.

Include new test method in OrmLearnApplication with following steps

Call the delete method based on the country code during the add country hands on

Check in database if the country is deleted